Architektur, die sich selbst kontrolliert

– Architekturtests

Wie garantiere ich, dass unsere Architektur in sechs Monaten noch so aussieht wie heute?



Architekturtests in vier Kapiteln

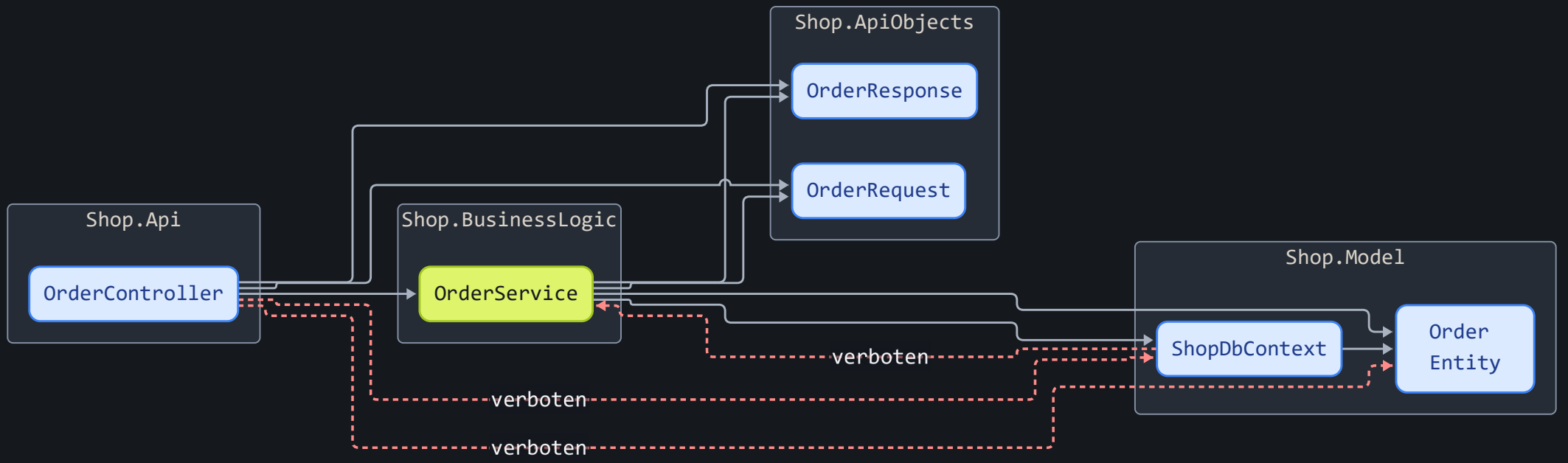
I Welches Problem wollen wir lösen?

II Welche Tools gibt es?

III Wie formulieren wir Architekturregeln und testen sie?

IV Meine Superstars unter den Architekturtests

Verbotene Zugriffe ausblenden



Durchgezogene Pfeile zeigen erlaubte Typabhängigkeiten, gestrichelte Pfeile zeigen verbotene Abhängigkeiten.

Die etablierte Struktur rostet.

Heute

Der `Controller` ruft den `Service` auf. Der `Service` ruft den `DbContext` auf.

Im nächsten Sprint

Der `Controller` greift direkt auf den `DbContext` zu. Geht schneller.

Die Unit-Tests bleiben grün. Die Architektur ist beschädigt.

Struktur prüfen, statt Verhalten.

Unit-Test

Berechnet der Service den richtigen Preis?

Testet Verhalten für konkrete Eingaben

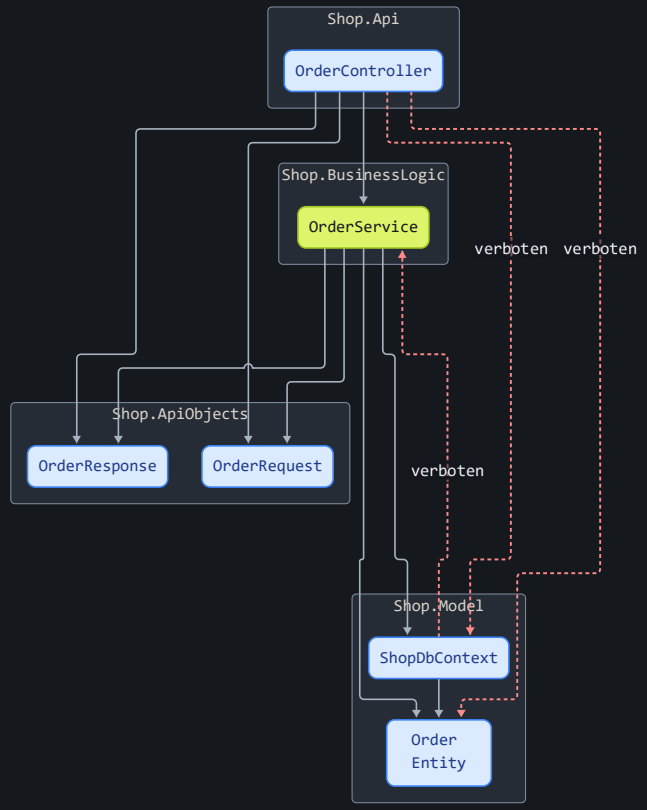
Architekturtest

Darf der Controller überhaupt auf den DbContext zugreifen?

Testet Struktur und Abhängigkeitsrichtung

Architekturentscheidungen werden zu ausführbaren Regeln.

4 Architekturregeln für unser Shop-Backend



	REGEL	WARUM?
1.	**Controller dürfen nicht auf den `DbContext` zugreifen.**	Datenbankzugriffe gehören in die BusinessLogic. So umgehen Controller keine Geschäftslogik.
2.	**Controller dürfen nicht auf Entities zugreifen.**	Die API soll nicht vom internen Datenmodell abhängen. Dafür verwenden wir DTOs aus ApiObjects.
3.	**Services dürfen nicht von Controllern abhängen.**	Die BusinessLogic bleibt unabhängig von der API und wiederverwendbar.
4.	**Entities dürfen nicht von API oder BusinessLogic abhängen.**	Das Datenmodell bleibt unabhängig von höheren Schichten. API-Änderungen erzwingen so keine Änderungen an Entities.

Welche Bibliotheken gibt es?

WERKZEUG	WOFÜR ES SICH ANBIETET
NetArchTest.Rules	Kompakte Regeln über Typen und Abhängigkeiten
ArchUnitNET	Ein ausführlicheres Modell der Architekturbeziehungen
.NET Reflection	Eigene, eng begrenzte Typkonventionen

Heute: NetArchTest + xUnit als Testing Framework

Die Struktur festnageln.

```
[Fact]
public void BusinessLogic_Should_Not_Depend_On_Api()
{
    var assembly = typeof(OrderService).Assembly;

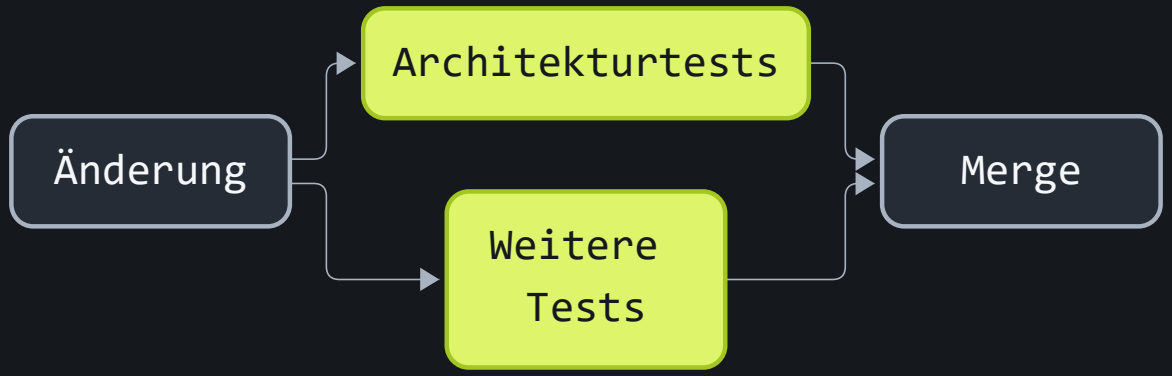
    var rule = Types.InAssembly(assembly)
        .That()
        .ResideInNamespace("Shop.BusinessLogic");

    Assert.NotEmpty(rule.GetTypes());

    var result = rule
        .ShouldNot()
        .HaveDependencyOn("Shop.Api").GetResult();

    Assert.True(result.IsSuccessful,
        "BusinessLogic must not depend on Api.");
}
```

Verstöße aufdecken.



- Wie andere Tests ausführen, im Pull Request verpflichtend machen.
- Regeln und Anwendungscode gemeinsam versionieren.

Meine Architekturtest-Superstars #1

```
/// <summary>
///     Ensures migrations use SqlWithExecute instead of the EF Core Sql method.
/// </summary>
[TestMethod]
public void Database_Migrations_Should_Only_Use_SqlWithExecute()
```

```
/// <summary>
///     Requires rollback implementations for migrations containing EF schema-change operations.
///     Data operations and raw SQL are excluded because SQL cannot reliably be classified as schema changes.
/// </summary>
[TestMethod]
public void Database_Migrations_Schema_Changing_Should_Have_Filled_Down_Methods()
```

Meine Architekturtest-Superstars #2

```
/// <summary>
///     Makes sure that all entities have the conventional Id/Uuid pair of properties,
///     e.g. "UserId" and "UserUuid" for the "User" entity.
/// </summary>
[TestMethod]
public void All_Entities_Should_Have_Conventional_Id_And_Uuid_Properties()
```

```
/// <summary>
///     Checks whether any Foreign Keys exists that are inferred by EF Core by their name.
/// </summary>
[TestMethod]
public void Model_ShouldNotHaveInferredForeignKeys()
```

Meine Architekturtest-Superstars #3

```
/// <summary>
///     Asserts that all enums in the ACOS.Central.Model.Enums namespace end with "Enum",
///     except for ApiErrorCode, whose established name is retained to avoid widespread changes.
/// </summary>
[TestMethod]
public void Model_Enums_In_Enums_Namespace_Should_End_With_Enum()
```

Take Home Messages

01

Erst die Basics absichern.

Backend: Namenskonventionen, Schichtengrenzen.

Frontend: Namenskonventionen, Schichtengrenzen, HTTP nur in Services, keine gegenseitigen Feature-Imports, etc.

02

Bekannte Fehlerquellen automatisieren.

Migrationen, EF-Core-Konfigurationen und API-Verträge prüfen. Besonders wertvoll sind Regeln, die bereits aufgetretene Fehler verhindern.

03

Regeln gehören in die CI.

Mit dem Anwendungscode versionieren, im Code Review prüfen und Verstöße vor dem Merge erkennen.